

이력서 초안 · 서버 개발자

측정으로 결정하는 백엔드 개발자

토스뱅크 · 토스플레이스 지원용 초안.

두 공고가 공통으로 요구하는 “문제 정의 → 시도 → 결과” 형식에 맞춰 두 프로젝트를 재서술했다.

표시 규칙



지금 제출 가능. 측정·코드로 확인된 사실



아직 미검증. 목표 문장이며 검증 전에는 이력서에서 뺀다

확인

내가 모르는 사실. 직접 채워야 함

△ 목록이 곧 다음 4주의 작업 목록이다. 검증되면 표시만 지운다.

0 먼저 채워야 하는 것

이 네 개가 비면 서류의 앞부분이 성립하지 않는다.

항목	왜 필요한가
확인 기간 경력 연차 · 재직	서류의 첫 판단 기준
확인 범위 회사 · 도메인 공개	"유통업 ERP" 까지 쓸지, "신규 대규모 시스템" 으로 돌지
확인 실무 시스템 규모	일 주문 건수 · 테이블 수 · 동시 사용자. 두 JD 가 "대규모 트래픽" 을 직접 묻는다
확인 개선 실무에서 직접 한	사이드 프로젝트만으로 "기존 대비 나은 결과" 를 채우기는 약하다

1 요약과 핵심 역량

✓ 통념을 측정으로 검증한 뒤 결정하는 백엔드 개발자.

신규 시스템의 DBMS 선택 근거를 직접 벤치마크로 검증했고(주문 607만 행 규모),
그 과정에서 내 측정이 드라이버 비대칭으로 오염된 것을 스스로 적발해 공정화했다.
결정은 코드가 아니라 문서로 남긴다 - 버린 안과 재검토 조건까지 함께.

첫 문장에 "측정" 을 놓았다. 두 JD 가 공통으로 요구하는 것이 문제 정의 → 시도 → 결과 이고, 이 사람의 일관된 패턴 이 그것이다.

역량	근거
✓ Kotlin/Java + Spring 서버 개발	필수 요건. 실무 + 정산어택
✓ 데이터 모델과 API 설계	엔드포인트 24개 · 오류 코드 31종을 구현 전에 계약으로 확정
✓ 성능 근거를 실행계획으로 증명	EXPLAIN ANALYZE 로 "왜 빠르냐" 까지 규명
✓ 정합성 · 격리수준	동일 격리수준에서 lost update 재현·차단을 실측
✓ 측정 설계 능력	드라이버 비대칭 오염 적발 → 공정 측정으로 교정
✓ 결정 기록	ADR 8건. 버린 안과 재검토 조건 포함
△ 동시성 제어	설계 완료, 부하 검증 예정
확인 인 대규모 트래픽 운영	실무 규모를 채워야 성립

2 프로젝트 A — 신규 시스템 DBMS 선택을 벤치마크로 검증

단독 설계·구현·측정 · 실무 의사결정을 사이드 프로젝트로 검증

문제

사내에서 신규 대규모 시스템 도입과 기존 시스템 마이그레이션을 진행 중이었고, 라이브 전이라 DBMS 를 다시 고를 수 있었다. 관성적으로 쓰던 MySQL 대신 PostgreSQL 을 선택했지만, 그 근거가 "복잡 쿼리·정합성엔 PG 가 낫다더라" 는 통념에 머물러 있었다.

제약

- DBMS 는 한번 정하면 되돌리기 어렵다. 마이그레이션 비용이 선택 비용보다 크다
- 두 엔진을 공정하게 비교해야 한다 — 코어·버퍼·데이터가 다르면 아무 의미가 없다
- "PostgreSQL 이 다 이긴다" 는 결론은 신뢰할 수 없다. 트레이드오프가 나와야 한다

시도

- 현장의 통념 5가지를 참 / 거짓 / 조건부로 가리는 형태로 문제를 정의
- 동일 코어 · 동일 버퍼(각 6GB) · 동일 데이터(CSV) 로 대칭 구성
- 단순 OLTP 는 sysbench, 복합 쿼리는 EXPLAIN ANALYZE 로 "왜 빠른지" 까지 규명, 동시성·정합성은 Python 하니스 로 직접 재현
- 주문-재고-배송 풀필먼트 14테이블 · 주문 607만 · 주문상세 1,321만 행
- OCI(4 vCPU / 24GB) 에서 최종 측정, GitHub Actions → SSH → OCI 자동화 — 호스팅 러너는 트리거만, 측정은 전부 실제 하드웨어에서

1,321만 행

측정 데이터 규모
주문상세 기준

4.7×

7테이블 조인
18.1s vs 85.5s

69×

1,300만 행 집계
0.87s vs 59.6s

257건

MySQL lost update
PG 는 0건

결과

- ✓ 대규모 조인 7테이블 조인 PG 18.1s vs MySQL 85.5s 4.7×
PG = Parallel Hash Join(4워커) / MySQL = 행당 PK 조회 Nested Loop
→ 튜닝 문제가 아니라 아키텍처 한계임을 실행계획으로 확인
- ✓ 대규모 분석 1,300만 행 집계 PG 0.87s vs MySQL 59.6s 69×
scale 이 커질수록 격차가 벌어짐
- ✓ 정합성 (결정타) 동일 RR 에서 MySQL lost update 257건 vs PG 0건
MySQL 은 조용히 덮어쓰기(fail-silent) · DDL 롤백 불가
PG 는 직렬화 오류로 거부(fail-loud) · DDL 롤백 가능
- ✓ 반증도 함께 insert 꼬리지연 max 는 MySQL 이 안정적 (44ms vs PG 268ms)
→ "PG 가 다 이긴다" 가 거짓임을 스스로 기록

★ 이 프로젝트의 핵심 — 내 측정이 틀렸음을 스스로 찾아냈다

통념 ④ “PG 는 MVCC 라 동시 쓰기가 빠르다” 를 재던 중, 양쪽 드라이버가 비대칭(한쪽만 네이티브 C)이라 측정이 오염된 것을 적발했다. 양쪽 모두 네이티브 C 드라이버로 교체해 공정화한 결과 쓰기 격차가 $1.44\times \rightarrow 1.22\times$ 로 좁혀졌다.

그리고 통념의 인과 자체가 틀렸음을 확인했다 — MySQL 도 MVCC 다. PG 의 modest 한 우위는 MVCC 때문이 아니고, 그 대가로 hot row dead tuple 34,169건의 bloat 을 진다.

측정 결과를 뒤집는 것보다 측정 방법을 의심하는 것이 어렵다. 적발·교정 과정을 별도 검증 로그 문서에 전부 기록했다.

결론을 맥락 의존으로 남긴 것이 이 프로젝트의 신뢰도다

직관으로 고른 PostgreSQL 이 우리 시스템 요구(복잡 쿼리·분석·정합성)에 한해서는 데이터로 뒷받침되는 선택이었다. 단순 키-값 트래픽 위주였다면 결론이 달랐을 수 있다.

3 프로젝트 B — 정산어택 (진행 중)

2026.08 ~ · 기획·설계·개발 단독 · 웹 + 모바일

문제

술자리 정산은 나눗셈 문제가 아니라 **기록 문제**다. 1차 5명, 2차 3명, 한 명은 낱알콜, 택시비는 2명 부담 — 인원수로 나누면 누군가는 반드시 손해를 본다. 그런데 **그 정보는 총무의 기억 속에만 있다.**

제약

- ▶ 금액은 1원도 틀리면 안 된다. 합계가 원금과 정확히 일치해야 한다
- ▶ 참여자에게 앱 설치를 요구할 수 없다 — 이탈하면 총무가 다시 전부 입력한다
- ▶ 무료 인스턴스 한 대에 앱·DB·관측 스택을 모두 올린다

시도와 결과

✓ **BigDecimal** 이 안전하다는 통념을 반증

랜덤 30,000건을 참조 구현과 대조 → **210건 (0.70%)** 금액 불일치
정밀도를 100자리로 올려도 사라지지 않음 = 정밀도가 아니라 표현의 문제
→ **BigInteger** 유리수로 교체 → 불일치 0건

✓ 합계 불변식을 코드 구조로 보장

대표결제자 몫을 나눗셈이 아니라 뺄셈으로 남김
`amounts[mainPayer] = grandTotal - amounts.values.sum()`
→ 보정 로직이 존재하지 않으므로 보정 버그도 존재할 수 없다

✓ 구현 전에 계약을 확정

엔드포인트 24개 · 오류 코드 31종 · 요청/응답 형식
계산 엔진의 **ErrorCode** enum 과 기계로 대조해 누락 0건 확인

✓ 결정을 기록으로 남김

ADR 8건 - 각 문서에 버린 안과 재검토 조건 포함
계산 엔진 739줄 / 테스트 741줄 / **46건** 전부 통과

▲ 동시성 제어를 부하로 검증

경합 4종을 성질별로 분리 (행 잠금 · 복합 유니크 · 조건부 UPDATE)
→ 동시 N 요청에서 정원 초과 0건 · p99 측정 예정

▲ 조회 쿼리 수를 고정

컬렉션 6개를 순진하게 로딩하면 N+1
→ `gathering_id` 로 따로 끊어 메모리 조립, 5~6 쿼리로 고정 측정 예정

설계 판단

판단	이유
계산 엔진을 DB-HTTP 없이 순수 함수로 격리	금액 정확성만 따로 검증하기 위해
계산 결과를 저장하지 않고 매번 호출	저장하면 "저장된 금액" 과 "지금 계산한 금액" 이 갈라진다
Gathering 을 애그리게이트 루트로, 그 행을 잠금 단위로	애그리게이트 경계 = 잠금 경계 = 트랜잭션 경계
브로커 대신 아웃박스 테이블	알림 하나 때문에 운영 대상을 늘리지 않는다. DB 커밋과 함께 원자적
비정규화 카운터를 쓰지 않음	갱신 경로가 5개라 하나만 빠뜨려도 정원 게이트가 조용히 틀린다

4 성장 — JD 가 직접 요구하는 항목

"어떤 성장을 생각하고 있고, 성장을 위해 어떤 노력을 하고 있는지"

✓ 통념을 측정으로 바꾸는 습관

"PG 가 낫다더라" → 5가지 통념을 참/거짓/조건부로 가림
 "BigDecimal 이면 된다" → 30,000건 대조로 반증

✓ 내 측정을 의심하는 습관

드라이버 비대칭 오염을 스스로 적발해 공정화 (1.44x → 1.22x)
 결론을 맥락 의존으로 남기고 반증 사례를 함께 기록

✓ AI 산출물을 검증하는 습관

생성된 설명 중 인과가 틀린 것을 적발한 로그를 별도 문서로 관리
 할루시 적발 1건 · 과잉 일반화 차단 2건 · 조건 명시 3건

✓ 결정을 문서로 남기는 습관

ADR 8건 · 버린 안과 재검토 조건을 함께 적어,
 나중에 근거를 모른 채 뒤집지 않게 만든다

▲ 구현력 보강

알고리즘·구현 훈련을 별도로 진행 중 (2026.08~10)
 → 설계는 되지만 빈 파일에서 코드를 만드는 속도가 부족하다는 자기 진단

마지막 항목은 서류에 넣지 않는 쪽을 권한다

약점을 먼저 말하는 것이 신뢰를 주기도 하지만, 서류 단계에서는 감점 요인이 될 수 있다. 면접에서 물으면 답하는 쪽이 낫다 — 그때는 "진단하고 조치 중" 이라는 강점이 된다.

세 번째 항목이 지금 가장 희소하다

AI 산출물의 인과가 틀린 것을 적발해 기록한 로그를 가진 지원자는 드물다. "AI 를 쓴다" 가 아니라 "AI 를 검증한다" 는 서술이라 차별점이 된다.

5 파생 질문 트리 — 정산어택

이력서 문장이 스터디 범위를 정의한다. 하루 1~2개씩 답을 채운다.

DB 프로젝트는 MySQL_PostgreSQL/report/면접_예상 질문.md 에 A~I 로 이미 있으므로, 여기서는 정산어택 문장에서 파생되는 것만 정리한다.

“유리수로 교체했다”

- BigDecimal 의 내부 표현은? $\text{unscaledValue} \times 10^{-\text{scale}}$ 인데 왜 1/3 을 담지 못하는가
- MathContext 를 100자리로 올려도 왜 해결되지 않는가
- double 은 왜 애초에 후보가 아닌가 — 이진 부동소수로 0.1 을 담지 못하는 문제
- BigInteger 연산 비용은? 분모가 커지면 어떻게 되는가 → 약분 시점
- 통화가 여러 개가 되면? 소수점을 쓰는 통화는 무엇이 달라지는가

“합계를 뺄셈으로 보장했다”

- 대표결제자가 음수가 될 수 있는 조건은? 그때 어떻게 처리하는가
- 반올림 단위를 100원으로 올리면 왜 음수가 더 쉽게 나오는가
- 이 방식의 단점은? 한 사람이 반올림 오차를 전부 흡수한다 → 공정한가

“애그리게이트 경계를 잠금 경계로 잡았다”

- SELECT ... FOR UPDATE 는 어떤 잠금인가. FOR SHARE 와 차이
- 왜 participant 가 아니라 gathering 을 잠그는가 → 없는 행은 잠글 수 없다
- MVCC 에서 일반 SELECT 는 왜 막히지 않는가
- 격리수준을 SERIALIZABLE 로 올리면 이 잠금이 필요 없어지는가 → 그 대가는
- 데드락이 나는 조건과 회피 규칙 → 잠금 순서 고정
- 낙관적 락(@Version)을 왜 안 썼는가 → 재시도를 참여자에게 요구하게 된다

“복합 유니크로 중복 참여를 막았다”

- 애플리케이션의 if (exists) throw 가 왜 불충분한가
- PostgreSQL 의 UNIQUE 는 NULL 을 어떻게 취급하는가 → 널 허용 시 제약이 무력화
- 제약 위반 예외를 어느 계층에서 도메인 오류로 변환해야 하는가
- 유니크 인덱스가 잠금에 미치는 영향

“조건부 UPDATE 로 상태 경합을 처리했다”

- UPDATE ... WHERE status = 'COLLECTING' 이 원자적인 이유
- 영향 행 수 0 을 오류로 볼 것인가 정상 종료로 볼 것인가 → 사용자 맥락에 따라
- 이것과 낙관적 락의 차이는 본질적으로 무엇인가

“브로커 대신 아웃박스를 썼다”

- 아웃박스가 해결하는 문제는? → DB 커밋과 메시지 발행의 원자성
- 아웃박스는 Kafka 의 대체인가 보완인가 → 보통 보완. 확장 경로는 Outbox → CDC → Kafka
- 발행 실패 시 중복 발행이 생기는데 소비자 멍등성은 어떻게 보장하는가
- Kafka 를 넣는다면 파티션 키는? gatheringId. 애그리게이트 경계와 같아야 하는 이유
- 파티션을 늘리면 순서 보장이 어떻게 깨지는가

“무료 인스턴스 한 대에 전부 올린다”

- 앱·DB·Prometheus·Loki·Grafana 의 메모리 배분 근거는? → 실측값 필요 (△)
- 컨테이너 메모리 상한이 없으면 무엇이 먼저 죽는가
- ARM64 크로스빌드가 필요한 이유

- 단일 VM 에서 K8s 를 쓰지 않은 이유

“요청 제한을 두 겹으로 나눴다”

- 엣지(Cloudflare)와 애플리케이션(Bucket4j)의 역할 분담 근거
- 연속 404 규칙을 엣지에서 못 하는 이유 → 토큰 유효성은 DB 를 봐야 한다
- 성공 응답을 세지 않는 이유 → NAT 뒤 다수 사용자
- 토큰 길이 12자-62종의 근거 → 대입 난이도 계산
- 인스턴스가 늘면 카운터가 왜 뚫리는가 → Redis 백엔드가 필요한 지점

6 4주 작업 목록 — △ 를 지우는 순서

주	작업	지워지는 △
1	스키마(Liquibase · ERD · 명세) + 서버 골격	—
2	엔드포인트 구현 · 프론트 연동 · 순진한 로딩 vs 조립 쿼리 수 측정	N+1
3	인프라 실측(메모리 배분) · Kafka 이력 스트림 · Redis 제한	메모리 배분 근거
4	부하 테스트 — 동시 요청 정원 초과 0건 · p99	동시성

§ 이 초안에서 의도적으로 빼놓은 것

빼 것	이유
K8s · Istio · ArgoCD	단일 VM 에 얹으면 “왜” 에 답이 없다. “왜 안 썼는지” 가 더 강한 답변이다
MongoDB · Elasticsearch	이 도메인에 자리가 없다. 억지로 넣으면 그 항목이 감점이 된다
RDS	DB 를 앱과 다른 클라우드에 두는 구조를 설명할 수 없다
사용자 수 · 다운로드 수	아직 출시 전이다. 출시 후 채운다

이력서 초안 · 수치는 두 저장소에서 직접 측정한 값이다 · 주문 607만/주문상세 1,321만 행 · core 739/741줄 · 테스트 46건 · ADR 8건